

RISC-V (VeeR EL2) Based SoC for Memory Built-In Self-Test and Fault Detection

Embedded memories are among the most defect-prone structures in a modern System-on-Chip (SoC). Once a device is deployed in the field, conventional logic scan chains cannot verify the memory subsystem. A related problem exists even in systems with built-in error correction: the VeeR EL2 core's Data Closely-Coupled Memory (DCCM) provides native SECDED ECC protection with in-line correction of single-bit errors, but the core's reference manual explicitly notes that no address information is captured for individual correctable errors—only a running count. Memory Built-In Self-Test (MBIST) is the intended mechanism to determine the exact location of a bad bit.

This project addresses both problems by designing a RISC-V based SoC using the VeeR EL2 core as its central processing unit. The SoC autonomously and periodically tests the DCCM for structural faults through a custom MBIST Controller, correlates those findings with run-time ECC activity via a firmware-mediated ECC Monitor, and reports results to an external host through UART and GPIO—without requiring continuous manual intervention or external test equipment.

The architecture standardizes on the AMBA AXI4 / AXI4-Lite protocol family throughout. The VeeR EL2 core complex provides 64-bit AXI4 interfaces for Instruction Fetch (IFU), Load/Store (LSU), and DMA access. The MBIST Controller operates as a 64-bit AXI4 master on the core's DMA Slave Port to directly inject and verify test patterns in DCCM without CPU pipeline stalls. All memory-mapped peripheral IP blocks (UART, System Timer, GPIO, Watchdog Timer, ECC Monitor, and MBIST Controller control/status registers) are implemented with native 32-bit AXI4-Lite slave interfaces attached to a unified AXI4 Crossbar Interconnect.

The document presents the complete SoC architecture including the processor core and its four AXI4 bus interfaces, the AXI4 crossbar interconnect and address decoder, the memory map, the complete signal list, detailed descriptions of every hardware block and register, control and data flow descriptions, an RTL design methodology, and a comprehensive verification and test plan.

Keywords: RISC-V, VeeR EL2, MBIST, ECC, SECDED, AXI4, AXI4-Lite, SoC, Fault Detection, Memory Test, DCCM, Hardware Self-Test, Crossbar Interconnect.

List of Tables

Table 1	SoC Block Summary
Table 2	Memory Map
Table 3	External Signals — Chip Boundary I/O
Table 4	Core-Level Internal Signals
Table 5	AXI4 and AXI4-Lite Bus Signals — Channel Breakdown
Table 6	MBIST Controller Registers and Signals
Table 7	ECC Monitor Registers and Signals
Table 8	Watchdog Timer Registers and Signals
Table 9	System Timer, UART, and GPIO Registers
Table 10	VeeR EL2 Core Complex Port List (Selected Signals)
Table 11	mdccmect CSR — DCCM Correctable Error Counter/Threshold Register
Table 12	Interrupt Priority Table
Table 13	MBIST Test Algorithm Summary
Table 14	Fault Detection Capability Summary
Table 15	Verification Test Plan Summary

1. Introduction

1.1 Background and Motivation

Semiconductor memories constitute a significant and growing fraction of the transistors on a modern System-on-Chip (SoC). Manufacturing defects, ageing mechanisms such as negative-bias temperature instability (NBTI) and hot-carrier injection, and cosmic-ray induced soft errors can all cause individual memory cells to fail. Unlike logic circuits, embedded memories cannot be fully exercised by conventional logic scan chains, making them among the most defect-prone and hardest-to-test structures in a design.

Memory Built-In Self-Test (MBIST) is the industry-standard technique for detecting structural memory faults. By running pre-defined test algorithms—such as March C- and Checkerboard—directly from on-chip hardware, MBIST can identify stuck-at faults, address-decode faults, and cell-coupling faults without requiring external test equipment. For systems that remain deployed in the field, the ability to run MBIST autonomously and periodically is particularly valuable: it can detect a developing hard fault before it causes a system failure.

A separate but related challenge arises with SECDED (Single-bit Error Correction / Double-bit Error Detection) ECC. Modern RISC-V cores such as the VeeR EL2 provide native SECDED ECC on their Data Closely-Coupled Memory (DCCM). While single-bit errors are corrected transparently in hardware, the core only maintains a running count of correctable errors—it does not record the address of each individual correction. This makes it impossible for software alone to distinguish between a transient soft error (which need not be acted upon) and a developing hard fault at a specific cell (which requires maintenance action). The core's own reference manual explicitly recommends MBIST as the tool to localise bad bits that the ECC counter alone cannot locate.

1.2 Problem Statement

Two problems are addressed by this project:

1. Embedded memories in deployed SoC-based systems cannot be verified through conventional scan chains. Most systems have no on-chip mechanism to confirm that their memory subsystem is functioning correctly after deployment.
2. The VeeR EL2 core's DCCM provides native SECDED ECC with in-line single-bit correction, but per the VeeR EL2 Programmer's Reference Manual (PRM), no address information is captured for individual correctable errors—only a running count is maintained in the mdccmect CSR (0x7F2). The correctable-error counter alone cannot distinguish a transient soft error from a developing hard fault at a specific cell.

1.3 Proposed Solution

This project addresses both problems by building a RISC-V based SoC that:

- Uses the VeeR EL2 RV32IMC core (M-mode only) as its central processing unit.
- Provides autonomous, on-demand structural fault detection in DCCM via a custom MBIST Controller that operates through the core's DMA Slave Port, independent of the CPU's normal load/store traffic.
- Correlates the core's native DCCM ECC activity with MBIST scan history through a firmware-mediated ECC Monitor—without duplicating the core's built-in correction hardware.
- Supervises firmware liveness with a Watchdog Timer, forcing recovery if self-test execution hangs.
- Schedules self-test execution automatically using a System Timer.
- Reports test results and system status externally via UART and provides simple pass/fail indication via GPIO.
- Maintains a high-performance AXI4 / AXI4-Lite architecture throughout—leveraging 64-bit AXI4 for high-bandwidth core memory/DMA paths and native AXI4-Lite for peripheral registers, avoiding legacy bus bridge bottlenecks.

1.4 Design Objectives

The specific design objectives of this project are:

3. Integrate the VeeR EL2 RISC-V core (RV32IMC, M-mode only) as the central control unit.
4. Implement a custom MBIST Controller as a 64-bit AXI4 master on the DMA Slave Port, supporting Checkerboard and March C- test algorithms with configurable address range.
5. Implement an ECC Monitor peripheral that receives firmware-mirrored snapshots of the mdccmect CSR and correlates them with the active MBIST scan window to identify suspect DCCM addresses—without duplicating the core's native ECC hardware.
6. Implement a Watchdog Timer to supervise firmware liveness and force system reset if a self-test scan or firmware loop hangs.
7. Implement a System Timer to automatically schedule periodic self-test runs via the MBIST Controller's mbist_start register.
8. Implement UART for external result and status reporting to a Host PC.
9. Implement GPIO for simple pass/fail indication using LEDs and external switches.
10. Standardize on the AMBA AXI4 / AXI4-Lite protocol family to maximize memory test throughput, support pipelined burst transactions, and eliminate AHB back-to-back write turnaround penalties.
11. Ensure that every peripheral and control interface is accessible as a native AXI4-Lite slave at a defined, aligned 4 KB address boundary.

1.5 Scope

This document covers the complete architecture of the SoC: the processor core and its bus interfaces, the AXI4 crossbar interconnect, the custom MBIST Controller, the ECC Monitor,

the Watchdog Timer, the System Timer, the UART and GPIO peripherals, the memory subsystem, the control and data paths, the design methodology, and the verification plan.

Note: *Physical timing closure, FPGA bitstream generation, and actual silicon area/power numbers are outside the scope of this project. The design targets technology-independent synthesisable RTL.*

1.6 Document Organisation

The remainder of this document is organised as follows:

- Chapter 2: Technical Background — RISC-V ISA, VeeR EL2 core architecture, AXI4 and AXI4-Lite protocols, MBIST theory, and ECC theory.
- Chapter 3: Proposed SoC Architecture — Block diagram, block definitions, system overview.
- Chapter 4: Detailed Description of Each SoC Block — Functional description, interface, and role of every block.
- Chapter 5: Memory Subsystem — ICCM, DCCM, ECC protection, memory map.
- Chapter 6: Bus and Interconnect — AXI4 protocol, channel handshake mechanisms, crossbar interconnect, address decoder, and arbitration.
- Chapter 7: Signal List — Complete external and internal signal definitions.
- Chapter 8: Control Flow — Configuration, scheduling, interrupt-driven, and watchdog-driven control.
- Chapter 9: Data Flow — Instruction fetch, load/store, MBIST test pattern, and ECC correlation paths.
- Chapter 10: Design Methodology — Design approach, tools, and flow.
- Chapter 11: Verification and Test Plan — Test strategy, scenarios, and expected results.
- Chapter 12: Limitations and Future Enhancements.
- Chapter 13: Conclusion.
- References: Bibliography.

2. Technical Background

2.1 RISC-V Instruction Set Architecture

RISC-V is an open, royalty-free instruction set architecture (ISA) based on established reduced instruction set computing principles. Its modular design consists of a mandatory base integer ISA (I extension, either 32-bit RV32I or 64-bit RV64I) plus optional standard extensions. The key properties that make RISC-V attractive for embedded SoC design are:

- Open and royalty-free specification maintained by RISC-V International.
- Fixed 32-bit instruction encoding in the base ISA with a 16-bit compressed (C) extension.
- Clean privilege architecture: Machine (M), Supervisor (S), and User (U) modes.
- Standardised Control and Status Registers (CSRs) for exception handling, timers, and counters.
- Well-defined memory model and memory-mapped I/O conventions.

This project uses the RV32IMC profile: 32-bit base integer (I), integer multiply/divide (M), and compressed instructions (C). The core operates in Machine mode (M-mode) only—the simplest privilege configuration, appropriate for embedded microcontroller-class applications.

2.2 VeeR EL2 Core

The VeeR EL2 is an open-source, M-mode-only, 32-bit RISC-V core developed by Western Digital and donated to the CHIPS Alliance. It implements RV32IMC with additional Zba, Zbb, Zbc, and Zbs bit-manipulation sub-extensions. Its key microarchitectural characteristics relevant to this project are:

2.2.1 Pipeline

The VeeR EL2 implements a 4-stage, scalar, in-order pipeline with the stages named Fetch (F), Decode (D), Execute/Memory (X/M), and Retire (R). There are two stall points in the pipeline (Fetch and Decode). The branch mispredict penalty is 1 or 2 cycles. The design supports a load-to-use latency of 1 cycle for smaller closely-coupled memories and 2 cycles for larger memories.

2.2.2 Core Complex

The core complex includes the core pipeline (IFU, DEC, EXU, LSU), optional ICCM and DCCM tightly-coupled memories, an optional instruction cache (I-cache), and an integrated Programmable Interrupt Controller (PIC). The core complex exposes four independent system bus interfaces configured as 64-bit AXI4:

- IFU Master: Instruction fetch; 64-bit read-only AXI4 master targeting ICCM and system instruction memory.

- LSU Master: Load/store data accesses; 64-bit read/write AXI4 master targeting DCCM, system memory, and memory-mapped peripherals.
- Debug Master (sb_axi): System-bus access port for the RISC-V Debug Module; 64-bit AXI4 master used by JTAG-attached debuggers.
- DMA Slave Port (dma_axi): 64-bit AXI4 slave port; allows external AXI4 masters (e.g., the MBIST Controller) to read/write ICCM and DCCM directly, independent of the CPU.

This project configures all four interfaces as 64-bit AXI4. The target clock frequency is 600 MHz at a 16 nm ASIC technology node; a lower frequency is used for FPGA or simulation.

2.2.3 DCCM ECC

The DCCM is protected by a Hamming (39,32) SECDED code: 32 data bits and 7 check bits per word. The hardware automatically corrects single-bit errors in-line on every DCCM read. Double-bit errors are detected but not corrected; they produce an AXI slave error response (RRESP = 2'b10 / SLVERR) on the read channel and an NMI to the core. The correctable-error counter mdccmect (CSR 0x7F2) is incremented on every retired correctable error; when the count reaches the programmed threshold (thresh field, bits [31:27]), a correctable-error local interrupt (mcause = 0x8000_001E) is signalled. Crucially, no address is recorded per correctable error—only the running count.

2.2.4 DMA Slave Port

The DMA Slave Port allows external AXI4 masters to read/write ICCM and DCCM over a 64-bit interface. Write accesses must be 32-bit or 64-bit aligned with full byte strobes (WSTRB = 8'hFF for 64-bit transfers) to avoid read-modify-write cycles for ECC generation. Core accesses have higher priority than DMA accesses when the DMA FIFO is not full; however, to prevent starvation, the DMA controller may periodically request a bubble. The maximum additional wait before a DMA access to DCCM is granted is 4 core clock cycles (plus up to 8 cycles of bubble-request latency, controlled by the dqc field of the mfdc CSR). There are no ordering guarantees between core and DMA accesses to the same address.

2.3 AXI4 and AXI4-Lite Protocols

AMBA AXI4 is a high-performance, point-to-point and crossbar-oriented bus protocol defined by ARM. It provides five independent, unidirectional channels for all transactions: Write Address (AW), Write Data (W), Write Response (B), Read Address (AR), and Read Data (R). Every channel uses an identical two-way VALID/READY handshake: transfer occurs on a clock edge when both VALID and READY are asserted. Because address and data phases are decoupled, multiple outstanding transactions and pipelined bursts are supported naturally. Read and write paths are fully independent, allowing simultaneous full-duplex transactions without bus turnaround penalties. Response codes (OKAY=2'b00, EXOKAY=2'b01, SLVERR=2'b10, DECERR=2'b11) indicate transfer success or slave/decode fault conditions. AXI4-Lite is a subset of AXI4 designed for memory-mapped register access: it uses single

transfers (burst length = 1 beat, no burst control signals) and fixed 32-bit data widths, reducing gate count while maintaining full compatibility with the AXI4 crossbar.

Note: Adopting AXI4 overcomes a key limitation documented in the VeeR EL2 PRM errata for AHB-Lite: on AHB-Lite, back-to-back LSU write transactions required an idle cycle between transfers. With AXI4, the decoupled Write Address (AW) and Write Data (W) channels allow continuous, back-to-back streaming writes without forced idle cycles, significantly accelerating both CPU stores and MBIST pattern generation sweeps.

2.4 MBIST Theory

Memory Built-In Self-Test (MBIST) applies deterministic write-and-read-back test patterns to a memory array to detect structural faults. The two algorithms used in this project are:

2.4.1 Checkerboard Test

The Checkerboard algorithm writes alternating 0x55555555/0xAAAAAAAA patterns to every address in the test region, reads back each address, and compares the read data with the written pattern. It is effective at detecting stuck-at-0 and stuck-at-1 faults.

2.4.2 March C- Algorithm

March C- is a more comprehensive algorithm that performs the following sequence over the test address range:

- $\uparrow \{w0\}$ — write 0 to all cells in ascending address order
- $\uparrow \{r0, w1\}$ — read 0 then write 1, ascending
- $\uparrow \{r1, w0\}$ — read 1 then write 0, ascending
- $\downarrow \{r0, w1\}$ — read 0 then write 1, descending
- $\downarrow \{r1, w0\}$ — read 1 then write 0, descending
- $\uparrow \{r0\}$ — read 0 in ascending order

March C- detects stuck-at faults, address-decode faults, and transition faults. It has linear complexity $O(n)$ and is the standard algorithm for production MBIST.

2.4.3 MBIST Limitations with ECC Memory

Because DCCM reads pass through the core's in-line ECC correction logic, a single stuck-at bit does not produce a data mismatch on a simple write/read-back/compare test: the core silently returns the corrected value to the MBIST Controller. Therefore:

- Double-bit (uncorrectable) faults and address-decode faults are directly detectable by the MBIST Controller: they produce $RRESP = 2'b10$ (SLVERR) on the DMA Slave Port read data channel.

- Single-bit hard faults are inferred indirectly: an abnormally fast rise in the mdccmect count while the MBIST scan covers a given address range is the signal the ECC Monitor uses to flag a developing hard fault at that location.

2.5 ECC Theory

The DCCM uses a Hamming-based SECDED code. The (39,32) code stores 32 data bits with 7 check bits, producing a 39-bit codeword. The minimum Hamming distance of 4 allows single-bit error correction and double-bit error detection. The check bits are computed as XOR combinations of specific data bit positions during a write and verified on every read. If exactly one bit is in error, the syndrome identifies the bit position and the hardware flips it in-line before returning data to the CPU. If two bits are in error, the syndrome is non-zero but does not correspond to any single-bit error location, signalling an uncorrectable error.

An important property: DMA write accesses to DCCM always overwrite entire 32-bit words and their corresponding ECC bits. ECC bits are never checked on DMA writes. Therefore, the MBIST Controller's write phase always produces correct ECC for the written pattern, and only the read phase—processed through the core's ECC correction—can encounter a correctable error from a pre-existing stuck-at bit.

3. Proposed SoC Architecture

3.1 System Overview

The SoC is organised around the VeeR EL2 RISC-V core, which exposes four independent AXI4 interfaces (IFU master, LSU master, Debug master, and DMA slave port)...

The MBIST Controller uses the DMA Slave Port to gain direct, CPU-independent access to DCCM. It operates as a 64-bit AXI4 master driving write and read patterns through the core's DMA port, while firmware supervises the process via the controller's AXI4-Lite slave registers. The ECC Monitor observes the firmware-relayed ECC error counter, correlating it with the MBIST test progress. The Watchdog Timer monitors firmware execution and forces a system reset via `rst_l` if firmware fails to service it within the programmed period. The System Timer provides periodic MBIST scan triggers. A UART interface provides serial reporting of test status and results, and GPIO lines drive external status LEDs.

3.2 Block Diagram

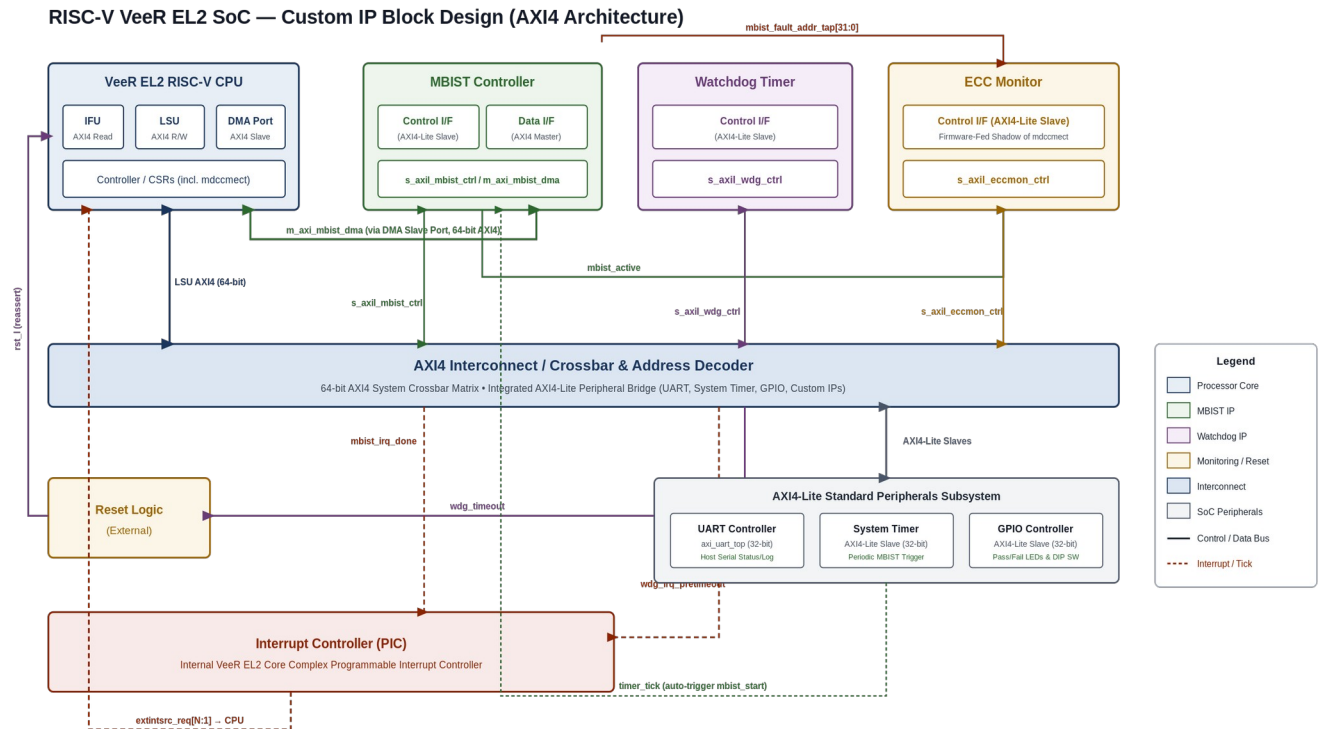


Figure 1: SoC Block Diagram — VeeR EL2 Core Complex, AXI4 Interconnect / Crossbar, and All Peripheral IPs

Note: Blue = core / mandatory infrastructure. Red = fault-detection IPs (MBIST, ECC Monitor). Purple = reliability IP (Watchdog). Grey = standard peripherals. The MBIST Controller drives the DMA Slave Port as a 64-bit AXI4 master; all memory-mapped control and status registers are accessed as 32-bit AXI4-Lite slaves routed through the shared AXI4 crossbar interconnect.

3.3 Block Summary

Table 1 lists all SoC blocks, their role, and their bus interface type.

Block	Role	Bus Interface
VeeR EL2 Core Complex	Central 32-bit RISC-V processor (RV32IMC, M-mode)	4 × AXI4 master/slave ports (64-bit data)
ICCM	Instruction Closely-Coupled Memory — core-local	Core-internal (not bus slave)
DCCM	Data Closely-Coupled Memory — native SECDED ECC	Core-internal + DMA Slave Port
AXI4 Interconnect / Crossbar	Address decoder and request router	N masters → M slaves (AXI4 64-bit crossbar + AXI4-Lite bridge)
MBIST Controller	Structural fault detection via DCCM test patterns	AXI4 master (64-bit DMA Port) + AXI4-Lite slave (32-bit registers)
ECC Monitor	Firmware-fed shadow of mdccmect; correlates ECC events with MBIST windows	AXI4-Lite slave (32-bit registers only)
Watchdog Timer (WDT)	Firmware liveness supervision; asserts rst_l on timeout	AXI4-Lite slave (32-bit registers)
System Timer	Periodic mbist_start trigger; 32-bit countdown	AXI4-Lite slave (32-bit registers)
UART	Serial host communication (result / status reporting)	AXI4-Lite slave (32-bit, matches axi_uart_top)
GPIO	Pass/fail LED outputs; external switch inputs	AXI4-Lite slave (32-bit registers)

Table 1: SoC Block Summary

4. Detailed Description of Each SoC Block

4.1 VeeR EL2 RISC-V Core Complex

The VeeR EL2 is the sole processing element in the SoC. It executes firmware stored in ICCM, accesses DCCM and memory-mapped peripherals through its LSU interface, and controls every aspect of system operation including MBIST scheduling, ECC monitoring, and external reporting.

Key characteristics relevant to this SoC:

- ISA: RV32IMC with Zba/Zbb/Zbc/Zbs bit-manipulation sub-extensions.
- Privilege: M-mode only — no Supervisor or User mode.
- Pipeline: 4-stage scalar in-order (Fetch, Decode, Execute/Memory, Retire).
- Load-to-use latency: 1 cycle for smaller memories, 2 cycles for larger memories.
- Target frequency: 600 MHz at 16 nm ASIC; lower for FPGA or simulation.
- Bus interfaces: IFU Master, LSU Master, Debug Master, DMA Slave Port — all 64-bit AXI4.
- Internal memories: ICCM (instruction), DCCM (data, native SECDED ECC).
- Interrupt controller: integrated PIC supporting up to 255 external interrupt sources with 15 priority levels.
- Debug: RISC-V Debug spec compliant via JTAG (TCK, TMS, TDI, TDO, TRST_n).
- Reset: rst_l — active-low, async assert, sync de-assert; rst_vec[31:1] supplies the boot vector.
- NMI: nmi_int input — edge-triggered, asynchronous; nmi_vec[31:1] supplies the NMI handler vector. NMIs are fatal; the core must be reset after an NMI.

The core's mbist_mode pin (when asserted) enables the core's own foundry-level structural MBIST for ICCM/DCCM by bypassing ECC logic. This is entirely distinct from the custom MBIST Controller IP built in this project and is tied low during normal operation.

4.2 Instruction Closely-Coupled Memory (ICCM)

The ICCM stores the firmware executed by the VeeR EL2 core. It is accessed exclusively by the IFU Master interface for instruction fetch. The ICCM is a core-local memory: it is not addressable as a generic bus slave but can be written through the DMA Slave Port (via 64-bit AXI4 writes) for firmware loading purposes.

ICCM is protected by SECDED ECC. Single-bit errors during instruction fetch are corrected in-line; the ICCM performs an RFPC flush and re-fetches from ICCM after writing the corrected data back. Double-bit errors cause an instruction access fault exception. The correctable error count is tracked in the miccmect CSR (0x7F1).

- Base address: 0x0004_0000 (example; build-time configurable).
- Size: 128 KB (example; valid sizes: 4/8/16/32/64/128/256/512 KB).
- Protection: SECDED ECC (39,32).
- DMA write rule: must be 32-bit or 64-bit aligned and sized.

4.3 Data Closely-Coupled Memory (DCCM)

The DCCM is the primary data memory for the SoC. It is the target of the MBIST Controller and the subject of all ECC monitoring activity.

The DCCM is accessed by three independent paths:

12. LSU Master (core load/store): every access is transparently ECC-checked and single-bit corrected in-line. The corrected data is written back to DCCM. The correction is invisible to software except through the incrementing mdccmect count.
 13. DMA Slave Port (MBIST Controller AXI4 master): the MBIST Controller drives write/read-back test patterns over 64-bit AXI4 channels. Reads from DCCM through this port also pass through the core's ECC correction logic, so single-bit faults are corrected before reaching the MBIST Controller. Double-bit faults produce RRESP = 2'b10 (SLVERR) on the read data channel.
 14. Debug Master: JTAG-attached debugger access for diagnostic read/write during development.
- Base address: 0x0008_0000 (example; build-time configurable).
 - Size: 128 KB (example; valid sizes: 4/8/16/32/48/64/128/256/512 KB).
 - ECC: Hamming (39,32) SECDED — 32 data bits + 7 check bits per 32-bit word.
 - Single-bit: corrected in-line; mdccmect incremented; corrected data written back.
 - Double-bit: not corrected; RRESP = 2'b10 (SLVERR) on bus; NMI to core (mcause = 0xF000_0000 or 0xF000_0001).

Note: The DCCM must be initialised before any load instruction is executed. Aligned word-sized stores followed by a fence instruction are the recommended initialisation method, since stores do not check ECC during the write phase.

4.4 MBIST Controller

The MBIST Controller is the primary fault-detection IP of this project. It operates as a 64-bit AXI4 master on the VeeR EL2 DMA Slave Port to inject test patterns into DCCM and read them back, independent of the CPU's normal load/store traffic.

From the CPU's perspective, the MBIST Controller is also an AXI4-Lite slave: firmware configures the test by writing to its control registers and reads results from its status registers across the 32-bit peripheral bus.

4.4.1 Direct Fault Detection Capability

Because DCCM reads are corrected in-line by the core's ECC hardware before data reaches the MBIST Controller, the controller can directly detect only:

- Double-bit (uncorrectable) faults: the DMA Slave Port read returns RRESP = 2'b10 (SLVERR), which the MBIST Controller records as a fault at the current address.
- Address-decode faults: a write to address A followed by a read from address A returns data that was actually written to a different address, producing a data mismatch.

Single-bit hard faults are inferred indirectly through the ECC Monitor (see Section 4.5).

4.4.2 Test Algorithms

The MBIST Controller supports two selectable algorithms:

- Checkerboard (mbist_algo_sel = 0b00): alternating 0x55/0xAA pattern; detects stuck-at faults.
- March C- (mbist_algo_sel = 0b01): six-element march sequence; detects stuck-at, transition, and address-decode faults.

4.4.3 Operational Interface

The MBIST Controller exposes the following memory-mapped registers at base address 0x4000_3000:

Signal / Register	Access	Direction	Description
mbist_start	W	CPU-written	Write 1 to trigger a self-test run. Auto-clears.
mbist_algo_sel[1:0]	R/W	CPU-written	0b00 = Checkerboard, 0b01 = March C-
mbist_addr_start[31:0]	R/W	CPU-written	Start address of the DCCM region to test
mbist_addr_end[31:0]	R/W	CPU-written	End address of the DCCM region to test
mbist_busy	R	Status reg	Set while a self-test scan is in progress
mbist_done	R	Status reg	Set when the current scan completes
mbist_pass_fail	R	Status reg	0 = pass, 1 = directly-detected fault present
mbist_fault_addr[31:0]	R	Status reg	Address of the first directly-detected fault
mbist_fault_count[15:0]	R	Status reg	Count of directly-detected faults

Table 6: MBIST Controller Registers and Signals

4.4.4 Point-to-Point Signals

In addition to its AXI4-Lite slave registers, the MBIST Controller drives two point-to-point signals:

- mbist_active (output → ECC Monitor, Watchdog Timer): asserted while a self-test scan is running.
- mbist_fault_addr_tap (output → ECC Monitor, read-only): latched current test address/window for correlation.
- mbist_irq_done (output → PIC extintsrc_req): interrupt to the core when a scan completes.

4.5 ECC Monitor

The ECC Monitor is a firmware-fed observer peripheral. It has no independent path to DCCM and drives no signal back to the MBIST Controller or Watchdog. It can be disabled or

removed with no impact on MBIST or Watchdog functionality—only the fault-correlation reporting is lost.

The core's mdccmect CSR is accessible only to the core itself via CSR instructions; it is not readable over the AXI bus. The ECC Monitor bridges this gap through a firmware-mediated path:

15. On a correctable-error local interrupt (mcause = 0x8000_001E), the firmware ISR reads mdccmect via a CSR instruction.
16. The ISR writes the count and threshold values into ecc_mon_shadow_count and ecc_mon_shadow_thresh over the AXI4-Lite bus, and sets ecc_mon_event.
17. The ECC Monitor compares the timing of shadow-count increments against the MBIST Controller's current test address window (via mbist_active and mbist_fault_addr_tap) and populates ecc_mon_suspect_addr with the MBIST window address range active at the time of the increment.

The ECC Monitor exposes the following registers at base address 0x4000_4000:

Signal / Register	Access	Direction	Description
ecc_mon_enable	R/W	CPU-written	Enables monitor logic
ecc_mon_shadow_count[26:0]	R/W	CPU-written	Firmware mirror of mdccmect.count
ecc_mon_shadow_thresh[4:0]	R/W	CPU-written	Firmware mirror of mdccmect.thresh
ecc_mon_event	R/W1C	CPU-written	Firmware sets when correctable-error ISR fires
ecc_mon_suspect_addr[31:0]	R	Status reg	MBIST address range at time of last shadow-count increment
ecc_mon_status	R	Status reg	Flags: correctable_seen, mbist_correlated

Table 7: ECC Monitor Registers and Signals

Note: The ECC Monitor input signals mbist_active and mbist_fault_addr_tap are inputs to the Monitor from the MBIST Controller. The Monitor drives no output signals back to any other hardware block.

4.6 Watchdog Timer

The Watchdog Timer (WDT) supervises firmware liveness. Firmware must periodically write to the wdg_kick register within the configured timeout period. If it fails to do so—for example, because a self-test scan or firmware loop has hung—wdg_timeout is asserted and the SoC's external reset generation logic reasserts rst_l, forcing a system reset.

The wdg_status register latches the timeout_occurred flag across the reset, enabling post-mortem firmware inspection after recovery.

Signal / Register	Access	Direction	Description
wdg_enable	R/W	CPU-written	Enables the watchdog countdown
wdg_load[31:0]	R/W	CPU-written	Timeout and reload value (in clock cycles)
wdg_kick	W	CPU-written	Write any value to reset the countdown (pet the watchdog)
wdg_count[31:0]	R	Status reg	Current countdown value

			(diagnostic read)
wdg_timeout	—	Internal → reset logic	Asserted on countdown expiry; drives rst_l reassertion
wdg_status	R	Status reg	Bit 0: timeout_occurred — latched across reset for post-mortem
wdg_irq_pretimeout	—	Output → PIC	Early-warning interrupt before timeout fires

Table 8: Watchdog Timer Registers and Signals

4.7 System Timer

The System Timer provides periodic self-test scheduling. When enabled, it counts down from the programmed reload value and generates a timer_tick pulse on expiry. This pulse can be connected directly to the MBIST Controller's mbist_start register to auto-trigger a self-test run without requiring firmware intervention. Firmware may also trigger mbist_start manually by writing to the register.

Signal / Register	Access	Direction	Description
timer_en	R/W	CPU-written	Enables timer countdown
timer_reload[31:0]	R/W	CPU-written	Countdown reload value
timer_tick	—	Internal → MBIST Ctrl	Periodic pulse; triggers mbist_start

Table 9a: System Timer Registers

Note: The RISC-V standard timer interrupt (timer_int, CSR mtime/mtimecmp) must be provided by the SoC per the VeeR EL2 PRM. This System Timer is distinct from the standard RISC-V timer and is dedicated to scheduling MBIST scans.

4.8 UART

The UART provides byte-oriented serial communication between the SoC and an external Host PC. Firmware writes self-test results, ECC correlation status, and system health information to the uart_tx output. The Host PC can send commands or query status via uart_rx.

Signal / Register	Access	Direction	Description
uart_baud_div[15:0]	R/W	CPU-written	Baud rate divisor
tx_data[7:0]	W	CPU-written	Byte to transmit
rx_data[7:0]	R	Status reg	Last received byte
uart_tx_busy	R	Status reg	Transmitter busy
rx_valid	R	Status reg	Received byte available
framing_err	R	Status reg	Framing error on last received byte

Table 9b: UART Registers

4.9 GPIO

GPIO provides simple external I/O: pass/fail LED indication from MBIST results (gpio_out[7:0]) and external switch or control inputs (gpio_in[7:0]).

Signal / Register	Access	Direction	Description
gpio_dir[7:0]	R/W	CPU-written	Direction register: 1 = output
gpio_out_reg[7:0]	R/W	CPU-written	Output drive value
gpio_in_reg[7:0]	R	Status reg	Captured external input state

Table 9c: GPIO Registers

5. Memory Subsystem

5.1 Overview

The VeeR EL2 divides its 32-bit address space into sixteen contiguous 256 MB regions. Core-local memories (ICCM, DCCM) must be mapped to the same region and must not share that region with system bus-attached memories. All peripherals in this project are placed in region 0x4 (0x4000_0000–0x4FFF_FFFF) at 4 KB aligned addresses, consistent with the VeeR EL2 PRM's example map guidance.

5.2 Memory Map

Base Address	End Address	Size	Description
0x0000_0000	0x0003_FFFF	—	Reserved (per PRM example map)
0x0004_0000	0x0005_FFFF	128 KB (ex.)	ICCM — Instruction Closely-Coupled Memory, core-local
0x0006_0000	0x0007_FFFF	—	Reserved (per PRM example map)
0x0008_0000	0x0009_FFFF	128 KB (ex.)	DCCM — Data Closely-Coupled Memory, core-local, SECDED ECC
0x000A_0000	0x0FFF_FFFF	—	Reserved (rest of region 0x0)
0x1000_0000	0x1FFF_FFFF	256 MB	System memory-mapped CSRs (e.g., MSI registers — tied off)
0x4000_0000	0x4000_0FFF	4 KB	UART — host communication
0x4000_1000	0x4000_1FFF	4 KB	System Timer — periodic self-test scheduling
0x4000_2000	0x4000_2FFF	4 KB	GPIO — pass/fail indicator, external I/O
0x4000_3000	0x4000_3FFF	4 KB	MBIST Controller — BIST control, status, fault address/count
0x4000_4000	0x4000_4FFF	4 KB	ECC Monitor — firmware-fed mdccmect shadow, correlation status
0x4000_5000	0x4000_5FFF	4 KB	Watchdog Timer — firmware liveness supervision
0x4000_6000	0x4FFF_FFFF	—	Peripheral expansion (reserved)

Table 2: Memory Map

Note: ICCM and DCCM sizes are build-time parameters. 128 KB is the working example used throughout this document. Valid DCCM sizes: 4/8/16/32/48/64/128/256/512 KB. Valid ICCM sizes: 4/8/16/32/64/128/256/512 KB.

5.3 ICCM and DCCM Placement Rules

Per the VeeR EL2 PRM (Section 16.2.1), each memory or register block must be placed at a naturally aligned power-of-two address boundary relative to its own size ($\text{start_address} = n \times \text{size}$). For example, a 128 KB DCCM must be aligned to a 128 KB boundary. No two memories may overlap, and no memory may cross a region boundary.

Both ICCM and DCCM must be mapped to the same address region (region 0x0 in this design) and must not share that region with system bus-attached memories or peripherals. The peripheral region (0x4) is a separate region and does not conflict.

5.4 ECC Memory Initialisation

Memories with ECC protection must be initialised before any load instruction is executed. If the DCCM is uninitialised, a load following a store to the same DCCM address may return incorrect data. The recommended initialisation sequence is:

18. Write the entire DCCM range using aligned word-sized stores (which overwrite ECC bits without checking them).
19. Execute a FENCE instruction to ensure all stores have completed.
20. Only then execute load instructions from DCCM addresses.

Note: MBIST can be used for memory initialisation: writing any test pattern via the DMA Slave Port initialises ECC bits for every written word.

5.5 Access Properties

Memory Type	Cacheable	Side Effect	Notes
ICCM	No	No	Core-local instruction memory
DCCM	No	No	Core-local data memory
Memory-mapped control/status regs	No	Yes	Non-idempotent (may have side effects)
System SRAMs/ROMs	Yes	No	Idempotent; cacheable
I/O devices	No	Yes	Non-idempotent

Table 3a: Memory Type Access Properties (mrac configuration)

6. Bus and Interconnect

6.1 AXI4 and AXI4-Lite Protocol Overview

The SoC uses AMBA AXI4 (for core memory and DMA transfers) and AXI4-Lite (for peripheral register access) as its bus architecture. AXI4 separates all transactions into five independent channels: Write Address (AW), Write Data (W), Write Response (B), Read Address (AR), and Read Data (R). Each channel operates asynchronously and independently using a two-way VALID/READY handshake mechanism, allowing concurrent bidirectional data transfers and pipelined operation.

The primary AXI4 and AXI4-Lite signals comprising the five channels are:

Signal	Master Drives	Slave Drives	Description
awaddr[31:0]	Master	—	Write address bus: specifies target address for write transaction
awvalid	Master	—	Write address valid: indicates write address/control signals are valid
awready	—	Slave	Write address ready: slave can accept write address and control
awlen[7:0]	Master	—	Burst length: number of transfers in burst (0=1 beat, up to 256)
awsize[2:0]	Master	—	Burst size: transfer width (010=32-bit, 011=64-bit)
awburst[1:0]	Master	—	Burst type: 00=FIXED, 01=INCR (standard sequential), 10=WRAP
wdata[63:0]	Master	—	Write data bus (64-bit for Core/DMA, 32-bit for AXI4-Lite)
wstrb[7:0]	Master	—	Write byte strobes: lane byte enables (8'hFF for full 64-bit write)
wlast	Master	—	Write last beat indicator in a burst
wvalid	Master	—	Write data valid: master is driving valid write data and strobes
wready	—	Slave	Write data ready: slave can accept the write data
bresp[1:0]	—	Slave	Write response: 00=OKAY, 01=EXOKAY, 10=SLVERR, 11=DECERR
bvalid	—	Slave	Write response valid: slave is driving a valid response
bready	Master	—	Write response ready: master can accept the write response
araddr[31:0]	Master	—	Read address bus: specifies target address

			for read transaction
arvalid	Master	—	Read address valid: indicates read address/control signals are valid
aready	—	Slave	Read address ready: slave can accept read address and control
arlen[7:0]	Master	—	Burst length: number of transfers in read burst
arsize[2:0]	Master	—	Burst size: transfer width (010=32-bit, 011=64-bit)
arburst[1:0]	Master	—	Burst type: 00=FIXED, 01=INCR, 10=WRAP
rdata[63:0]	—	Slave	Read data bus (64-bit for Core/DMA, 32-bit for AXI4-Lite)
rresp[1:0]	—	Slave	Read response: 00=OKAY, 10=SLVERR (uncorrectable ECC), 11=DECERR
rlast	—	Slave	Read last beat indicator in a read burst
rvalid	—	Slave	Read data valid: slave is driving valid read data and response
rready	Master	—	Read data ready: master can accept read data

Table 5: AXI4 and AXI4-Lite Bus Signals — Channel Breakdown

Note: An uncorrectable DCCM double-bit error on a DMA Slave Port read is returned as RRESP = 2'b10 (SLVERR) directly to the MBIST Controller.

6.2 Four VeeR EL2 AXI4 Interfaces

The VeeR EL2 core complex exposes four 64-bit AXI4 interfaces. In this SoC they are configured with the following roles:

Interface	Role	Primary Target	Clock Enable Signal
IFU Master	AXI4 master (64-bit Read)	Instruction fetch from ICCM / System RAM	ifu_bus_clk_en
LSU Master	AXI4 master (64-bit R/W)	Load/store to DCCM, system RAM, and peripherals	lsu_bus_clk_en
Debug Master (sb_axi)	AXI4 master (64-bit R/W)	JTAG debugger system bus access	dbg_bus_clk_en
DMA Slave Port (dma_axi)	AXI4 slave (64-bit R/W)	MBIST Controller reads/writes to DCCM and ICCM	dma_bus_clk_en

Table 5a: VeeR EL2 AXI4 Interface Assignments

6.3 AXI4 Interconnect / Crossbar and Address Decoder

The AXI4 Interconnect implements a high-performance crossbar matrix with an integrated address decoder and protocol adapter. It decodes AWADDR and ARADDR, routing requests to the addressed target slave. For accesses directed to peripheral registers (0x4000_0000 to 0x4000_5FFF), an integrated AXI4-to-AXI4-Lite bridge downconverts 64-bit bursts to single 32-bit register transactions. Unmapped address accesses return DECERR (2'b11). Independent read and write channel arbiters schedule concurrent traffic between masters using fair round-robin scheduling.

Per the VeeR EL2 PRM (Section 2.14.3), core accesses have higher priority than DMA accesses when the DMA FIFO is not full. To prevent starvation, the DMA controller may periodically request a bubble. The maximum additional wait before a DCCM DMA access is granted is 4 core clock cycles after the bubble request (plus the dqc-controlled waiting time, default 7 cycles, before a bubble is requested).

6.4 Clock Enables

Each AXI4 interface has a dedicated clock enable signal (*_bus_clk_en: ifu_bus_clk_en, lsu_bus_clk_en, dbg_bus_clk_en, dma_bus_clk_en) provided by the SoC. These signals control the system-bus-to-core clock ratio for each interface independently. Supported ratios are 1:1 through 1:8 (system bus runs at 1/N of the core clock). All four clock enables may be the same value or different values. The primary AXI bus clock is aclk, with active-low synchronous reset aresetn.

6.5 Asynchronous Signals

The following signals are asynchronous to the core clock and are synchronised internally by the core complex (two-stage synchroniser). Each signal must be asserted for at least two full core clock cycles to guarantee detection:

Signal	Direction	Description
extintsrc_req[N:1]	Input	External interrupt sources (via PIC)
soft_int	Input	Standard RISC-V software interrupt
timer_int	Input	Standard RISC-V timer interrupt
nmi_int	Input	Non-Maskable Interrupt
i_cpu_halt_req	Input	PMU halt request
i_cpu_run_req	Input	PMU run request
mpc_debug_halt_req	Input	MPC debug halt request
mpc_debug_run_req	Input	MPC debug run request
jtag_tck/tms/tdi/trst_n	Input	JTAG interface signals

Table 4a: Core Complex Asynchronous Signals

7. Signal List

Note: CSR-resident signals (*mdccmect*, *miccmect*, *micect*, etc.) are accessible only to the core via CSR instructions; they are not signals on the core's external port list and are not directly readable by any external hardware block, including the ECC Monitor IP.

7.1 External Signals (Chip Boundary I/O)

Signal	Direction	Width	Description
clk	Input	1	Core complex clock (600 MHz target at 16 nm; lower for FPGA/sim)
rst_l	Input	1	Active-low system reset (async assert, sync de-assert); may also be asserted by WDT recovery logic
rst_vec[31:1]	Input	31	Reset/boot vector — must point into ICCM or a valid mapped address
jtag_tck	Input	1	JTAG Test Clock (async)
jtag_tms	Input	1	JTAG Test Mode Select (async, sync to jtag_tck)
jtag_tdi	Input	1	JTAG Test Data In (async, sync to jtag_tck)
jtag_trst_n	Input	1	JTAG Test Reset (async)
jtag_tdo	Output	1	JTAG Test Data Out (async, sync to jtag_tck)
uart_tx	Output	1	Serial transmit → Host PC
uart_rx	Input	1	Serial receive ← Host PC
gpio_out[7:0]	Output	8	Pass/fail LED and status indicators
gpio_in[7:0]	Input	8	External switches and control inputs

Table 3: External Signals — Chip Boundary I/O

7.2 Core-Level Internal Signals

Signal	Direction	Width	Description
nmi_int	Core input	1	Non-maskable interrupt (edge-triggered, async); used for fatal D-bus/ECC double-bit conditions only
nmi_vec[31:1]	Core input	31	NMI handler vector (tied to a fixed internal address)
timer_int	Core input	1	Standard RISC-V timer interrupt; driven by SoC timer peripheral
soft_int	Core input	1	Software interrupt; tied to 0 in this single-core design
extintsrc_req[N:1]	Core input	N	External interrupt sources to PIC (MBIST done, WDT pre-timeout)
core_id[31:4]	Core input	28	Core ID, tied to a fixed constant; sources mhartid[31:4]
scan_mode	Core input	1	Logic scan enable; tied low for functional operation
mbist_mode	Core input	1	Enables core's own foundry MBIST; tied low during normal operation
ifu_bus_clk_en	Core input	1	IFU AXI4 interface clock enable
lsu_bus_clk_en	Core input	1	LSU AXI4 interface clock enable
dbg_bus_clk_en	Core input	1	Debug AXI4 interface clock enable
dma_bus_clk_en	Core input	1	DMA Slave Port AXI4 interface clock enable
dbg_rst_l	Core input	1	Active-low reset for the Debug Module only
o_cpu_halt_status	Core output	1	Core halted indication to PMU

o_debug_mode_status	Core output	1	Core in Debug Mode indication
---------------------	-------------	---	-------------------------------

Table 4: Core-Level Internal Signals

7.3 Key Custom-IP Point-to-Point Signals

Signal	Connection	Width	Description
mbist_active	MBIST Ctrl → ECC Mon + WDT	1	Asserted while a self-test scan is running
mbist_fault_addr_tap	MBIST Ctrl → ECC Monitor	32	Read-only tap of MBIST current test address window for correlation
mbist_irq_done	MBIST Ctrl → PIC	1	Interrupt on scan completion (routed to extintsrc_req)
wdg_timeout	WDT → External Reset Logic	1	Asserted on countdown expiry; drives rst_1 reassertion
wdg_irq_pretimeout	WDT → PIC	1	Early-warning interrupt before timeout
timer_tick	System Timer → MBIST Ctrl	1	Periodic pulse; can auto-trigger mbist_start

Table 4b: Custom IP Point-to-Point Signals

7.4 DMA Slave Port (MBIST Controller Interface)

The DMA Slave Port uses the 64-bit AXI4 slave signal set described in Table 5b, with the MBIST Controller acting as the AXI4 master and the core's DCCM as the target. The complete DMA Slave Port port list consists of:

Signal	Direction	Width	Description
dma_axi_awid[ID-1:0]	Input (MBIST → Core)	ID	Write address transaction ID
dma_axi_awaddr[31:0]	Input	32	DMA write address (must be 32-bit or 64-bit aligned)
dma_axi_awlen[7:0]	Input	8	Burst length (number of data beats in write burst)
dma_axi_awsz[2:0]	Input	3	Burst size: 010 (32-bit word) or 011 (64-bit doubleword)
dma_axi_awburst[1:0]	Input	2	Burst type: 01 (INCR sequential addressing)
dma_axi_awvalid	Input	1	Write address valid handshake
dma_axi_awready	Output (Core → MBIST)	1	Core DMA write address ready handshake
dma_axi_wdata[63:0]	Input	64	Write data (64-bit doubleword; DCCM ECC generated)
dma_axi_wstrb[7:0]	Input	8	Write byte strobes (8'hFF for full 64-bit write)
dma_axi_wlast	Input	1	Write last beat flag

dma_axi_wvalid	Input	1	Write data valid handshake
dma_axi_wready	Output	1	Core DMA write data ready handshake
dma_axi_bid[ID-1:0]	Output	ID	Write response transaction ID
dma_axi_bresp[1:0]	Output	2	Write response: 00=OKAY, 10=SLVERR
dma_axi_bvalid	Output	1	Write response valid handshake
dma_axi_bready	Input	1	MBIST write response ready handshake
dma_axi_arid[ID-1:0]	Input	ID	Read address transaction ID
dma_axi_araddr[31:0]	Input	32	DMA read address (DCCM target)
dma_axi_arlen[7:0]	Input	8	Read burst length
dma_axi_arsize[2:0]	Input	3	Read burst size: 010 (32-bit) or 011 (64-bit)
dma_axi_arburst[1:0]	Input	2	Read burst type: 01 (INCR)
dma_axi_arvalid	Input	1	Read address valid handshake
dma_axi_arready	Output	1	Core DMA read address ready handshake
dma_axi_rid[ID-1:0]	Output	ID	Read data transaction ID
dma_axi_rdata[63:0]	Output	64	Read data from DCCM (in-line SECDED ECC corrected)
dma_axi_rresp[1:0]	Output	2	Read response: 00=OKAY, 10=SLVERR (double-bit ECC fault)
dma_axi_rlast	Output	1	Read last beat flag
dma_axi_rvalid	Output	1	Read data valid handshake
dma_axi_rready	Input	1	MBIST read data ready handshake

Table 5b: DMA Slave Port Signals (AXI4 64-bit)

Note: DMA write accesses always overwrite the full 32-bit word and its ECC bits. ECC is neither checked nor generated from partial data on DMA writes.

8. Control Flow

8.1 Overview

The control flow of the SoC describes how the processor configures and monitors all hardware blocks, how interrupts trigger actions, and how the Watchdog Timer enforces system recovery. There are four distinct control sub-paths:

- Configuration writes: CPU writes to control registers to start and configure each block.
- Status reads: CPU polls or interrupt-driven reads of status and result registers.
- Firmware-mediated CSR-to-peripheral bridge: the key architectural path for ECC correlation.
- Interrupt-driven and timer-triggered control: automatic scheduling and event handling.

8.2 Boot and Initialisation Sequence

After reset de-assertion, the core fetches the first instruction from the address on `rst_vec[31:1]`, which must point into ICCM. The firmware initialisation sequence is:

21. Initialise DCCM: write the entire DCCM range with aligned word stores; execute FENCE.
22. Configure GPIO: set direction register (`gpio_dir`) and clear outputs.
23. Configure Watchdog Timer: write `wdg_load` with the desired timeout period; set `wdg_enable`.
24. Configure System Timer: write `timer_reload`; set `timer_en` to enable periodic MBIST scheduling.
25. Configure UART: write `uart_baud_div`; enable UART.
26. Configure MBIST Controller: write `mbist_addr_start`, `mbist_addr_end`, `mbist_algo_sel`.
27. Configure ECC Monitor: write `ecc_mon_enable`; clear `ecc_mon_shadow_count`.
28. Enable interrupts: configure PIC priority and enable bits for MBIST-done, WDT pre-timeout, and correctable-error local interrupt.
29. Enable global interrupt handling: set `mie` in `mstatus`.
30. Begin main execution loop.

8.3 MBIST Scan Scheduling and Execution

Self-test scans can be triggered in two ways:

- Automatic (timer-driven): the System Timer generates a `timer_tick` pulse at the programmed period, which directly asserts `mbist_start`.
- Manual (firmware-driven): the CPU writes 1 to `mbist_start` at any time.

On triggering, the MBIST Controller:

31. Sets `mbist_busy`.

32. Asserts mbist_active (visible to ECC Monitor and Watchdog).
33. Runs the selected algorithm (Checkerboard or March C-) over the mbist_addr_start to mbist_addr_end range via the DMA Slave Port.
34. On completion: clears mbist_busy, sets mbist_done, updates mbist_pass_fail, mbist_fault_addr, and mbist_fault_count.
35. Asserts mbist_irq_done to the PIC, generating an external interrupt to the core.

The firmware interrupt handler reads the MBIST status registers, reports results via UART, drives GPIO pass/fail LEDs, and clears mbist_done.

8.4 Firmware-Mediated ECC Correlation Path

This is the most architecturally significant control path addition in this design. The core's mdccmect CSR (0x7F2) is not accessible over the system bus—only the core itself can read it via a CSR instruction. The ECC Monitor therefore cannot observe ECC activity directly. The firmware bridges this gap:

DCCM single-bit ECC error (during LSU load/store or DMA read)

|

v

mdccmect.count incremented by core hardware

|

v (when count reaches thresh)

correctable-error local interrupt (mcause = 0x8000_001E)

|

v

Firmware ISR executes:

1. CSRR t0, 0x7F2 // read mdccmect

2. SW t0, ecc_mon_shadow_count (via LSU AXI4 store to AXI4-Lite)

3. W1 ecc_mon_event register

|

v

ECC Monitor compares timing of shadow-count increment
against mbist_active and mbist_fault_addr_tap

|

v

`ecc_mon_suspect_addr` populated with MBIST window address

`ecc_mon_status.mbist_correlated` flag set

Note: The `mdccmect` correctable-error local interrupt (`mceip` bit 30 of `mip`) has higher priority than the standard RISC-V Software and Timer interrupts but lower priority than the External interrupt from the PIC (per the VeeR EL2 interrupt priority table).

8.5 Watchdog Timer Supervision

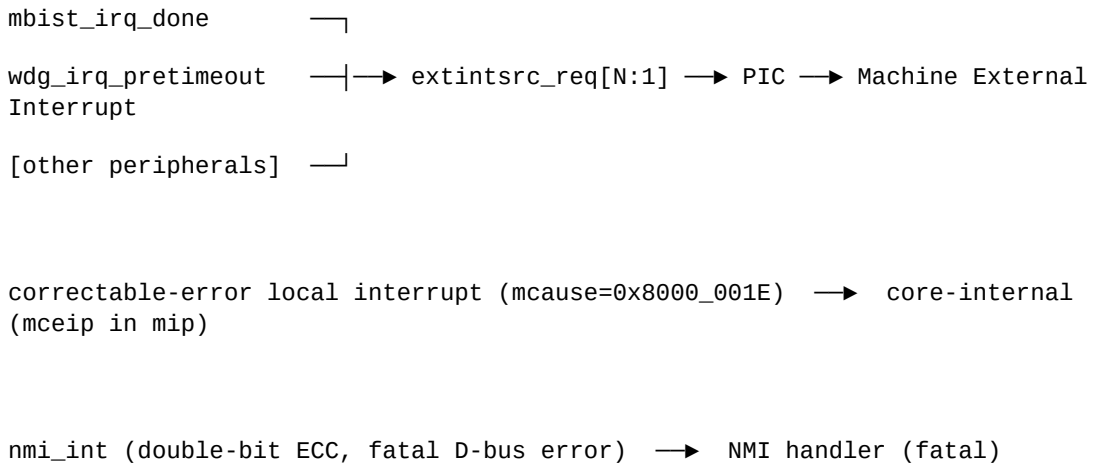
The Watchdog Timer runs continuously once enabled. Firmware must periodically write to `wdg_kick` before the countdown expires. If a self-test scan hangs (for example, due to a DMA transaction that never completes), or if a firmware loop stalls:

36. `wdg_count` reaches zero.
37. `wdg_timeout` is asserted.
38. External reset generation logic de-asserts and then re-asserts `rst_l` (synchronously), resetting the entire core complex except the Debug Module.
39. `wdg_status.timeout_occurred` is latched across the reset.
40. After reset de-assertion, firmware can read `wdg_status` to determine whether the previous boot was a watchdog recovery.

Note: NMI should not be used for Watchdog recovery. Per the VeeR EL2 PRM, NMIs are fatal — architectural state is lost and normal program execution cannot resume without a reset.

8.6 Interrupt Architecture

All interrupts are routed through the VeeR EL2's integrated PIC. Interrupt sources connect to `extintsrc_req[N:1]` pins. The PIC supports up to 255 sources with 15 priority levels, individually programmable per source.



The VeeR EL2 interrupt priority order (highest to lowest) is:

Priority	Interrupt Type	Usage in this SoC
1 (Highest)	Non-Maskable Interrupt (NMI)	Fatal; core must be reset
2	External interrupt (from PIC)	MBIST done, WDT pre-timeout, etc.
3	Correctable error local interrupt (<code>mceip</code>)	<code>mdccmect</code> threshold reached

4	Software interrupt	Tied to 0 — not used in this design
5	Timer interrupt	Standard RISC-V mtime/mtimecmp
6	Internal timer 0 (local)	Core-internal timer 0
7 (Lowest)	Internal timer 1 (local)	Core-internal timer 1

Table 12: Interrupt Priority Table

9. Data Flow

9.1 Overview

The SoC has five distinct data paths. Each is described below with the source, intermediate transformations, and destination.

9.2 Instruction Fetch Path

```
ICCM (core-local) → IFU pipeline (instruction fetch every cycle)
    ↓ (SECEDED ECC checked in-line)
    ↓ (single-bit error: corrected; RFPC flush; miccmect incremented)
    ↓ (double-bit error: instruction access fault exception)
Instruction decoder (DEC) → Execute (EXU/LSU) → Retire (R)
```

This path is entirely independent of the LSU, MBIST, and DMA paths. The IFU never accesses DCCM or any system bus slave.

9.3 Load/Store Data Path

```
CPU (EXU/LSU) → DCCM (core-local)
    ↓ (SECEDED ECC checked and corrected in-line on every read)
    ↓ (single-bit: corrected value returned; corrected data written back;
mdccmect++)
    ↓ (double-bit: RRESP=2'b10 SLVERR on LSU bus; NMI to core)
Corrected data returned to CPU registers
```

The load/store path also uses the LSU Master AXI4 interface to access all memory-mapped peripherals (MBIST Controller registers, ECC Monitor registers, WDT, System Timer, UART, GPIO) at addresses in the 0x4000_xxxx range via the AXI4-to-AXI4-Lite bridge.

9.4 MBIST Test-Pattern Path

```
MBIST Controller (64-bit AXI4 master)
    ↓ via DMA Slave Port (independent of CPU LSU traffic)
    ↓
Write phase: MBIST writes test pattern to DCCM address range
    (DMA write overwrites full 32-bit word + ECC bits; ECC not checked)
    ↓
Read phase: MBIST reads back from same address range
```

```

    (read passes through core ECC correction logic)

    (single-bit fault: silently corrected before reaching MBIST Controller)

    (double-bit fault: RRESP = 2'b10 SLVERR → mbist_pass_fail=1,
mbist_fault_addr latched)

    (address-decode fault: data mismatch → mbist_pass_fail=1)

↓

    mbist_done asserted → mbist_irq_done → PIC → CPU ISR → UART / GPIO
reporting

```

Note: Because the core's ECC correction is transparent to the MBIST Controller, single-bit hard faults appear as a rising mdccmect count, not as a data mismatch. They are therefore detected by the ECC Monitor correlation path (see Section 9.5), not directly by the MBIST Controller.

9.5 ECC Correlation Path (Firmware-Mediated)

This path is not a hardware data path; it is a firmware-mediated information relay that converts a CSR-resident counter into bus-accessible state:

```

DCCM single-bit ECC correction event

↓ (core hardware)

mdccmect.count incremented [CSR 0x7F2 – bus-inaccessible]

↓ (when count ≥ 2^thresh)

correctable-error local interrupt fired (mcause = 0x8000_001E)

↓ (firmware ISR)

CSRR reads mdccmect —► AXI4-Lite store to ecc_mon_shadow_count

↓ (ECC Monitor hardware)

Shadow-count increment timestamped against mbist_active + mbist_fault_addr_tap

↓

ecc_mon_suspect_addr = MBIST window address at time of increment

ecc_mon_status.mbist_correlated set

↓ (firmware reads)

CPU reports suspect DCCM address via UART

```

Figure 5: ECC Monitor — Firmware-Mediated Shadow Link and Address Correlation

9.6 UART and GPIO Reporting Path

Firmware result processing:

```

read mbist_pass_fail, mbist_fault_addr, mbist_fault_count

```


read ecc_mon_status, ecc_mon_suspect_addr

read wdg_status (post-mortem after watchdog recovery)

↓

write formatted result bytes to UART tx_data → uart_tx → Host PC

write mbist_pass_fail result to gpio_out_reg → gpio_out[7:0] → LEDs

9.7 Data Width Summary

Quantity	Width	Type	Notes
DCCM data word	32 bits	unsigned	Normal load/store; firmware and MBIST read/write data
DCCM ECC codeword	39 bits	—	Stored internally: 32 data + 7 check bits per word
mdccmect.count	27 bits	unsigned	Correctable error count in CSR 0x7F2[26:0]
mdccmect.thresh	5 bits	unsigned	Threshold selector; value i selects count[i] bit
ecc_mon_shadow_count	27 bits	unsigned	Firmware mirror of mdccmect.count
mbist_fault_addr	32 bits	unsigned	AXI address of first directly-detected fault
mbist_fault_count	16 bits	unsigned	Count of directly-detected faults
ecc_mon_suspect_addr	32 bits	unsigned	Inferred suspect address (MBIST window at time of ECC event)
AXI4 data bus (DMA)	64 bits	—	VeeR EL2 DMA Slave Port data width (64-bit AXI4)

Table 10a: Data Width Summary

10. Design Methodology

10.1 Overview

The design follows a top-down hardware/software co-design methodology. The system architecture is defined first at the block and interface level, followed by RTL implementation of each block, integration, and verification. All RTL is written in synthesisable SystemVerilog.

10.2 RTL Implementation Methodology

10.2.1 Design Language and Style

All synthesisable RTL is written in SystemVerilog. The following coding guidelines apply:

- Explicit signed/unsigned declarations and casts for all arithmetic operations.
- Explicit bit widths for all registers, wire declarations, and arithmetic intermediates.
- No floating-point in synthesisable logic.
- No # delays in synthesisable RTL.
- No testbench-only constructs (file I/O, \$display with non-simulation guards) in synthesisable modules.
- Synchronous active-low reset (rst_l) used consistently throughout, matching VeeR EL2 convention.
- No vendor-specific primitives in the synthesisable core; vendor-specific wrappers may be added separately if needed.

10.2.2 Block-Level Implementation Order

The recommended implementation order reflects block dependencies:

Stage	Block	Key Implementation Notes
1	VeeR EL2 core integration / wrapper	Connect four AXI4 interfaces (IFU, LSU, SB, DMA); tie off unused pins
2	AXI4 Interconnect and Address Decoder	Crossbar matrix; address decode; separate read/write routing; AXI4-Lite peripheral bridge; arbitration
3	ICCM and DCCM instantiation	Core-local memories at correct base addresses
4	Watchdog Timer	Countdown register; kick logic; timeout; wdg_status latch
5	System Timer	Auto-reload countdown; timer_tick generation
6	UART	Baud divisor; TX shift register; RX shift register; status flags
7	GPIO	Direction register; output register; input capture
8	MBIST Controller	DMA AXI4 master FSM (AW, W, B, AR, R channels); algorithm sequencer; result registers (AXI4-Lite)
9	ECC Monitor	Shadow register; correlation logic; suspect address register
10	Top-level integration	Connect all blocks; route interrupts to PIC; route WDT to reset logic

Table 14a: RTL Implementation Order

10.2.3 Clock and Reset Architecture

A single synchronous clock domain is used throughout. All flip-flops are clocked on the rising edge of `clk`. The `rst_l` signal is active-low, asynchronously asserted, and synchronously de-asserted (synchronised to `clk` before release). The Watchdog Timer recovery path generates a reset pulse by asserting `wdg_timeout`, which drives external reset generation logic that re-asserts `rst_l` for the required number of clock cycles.

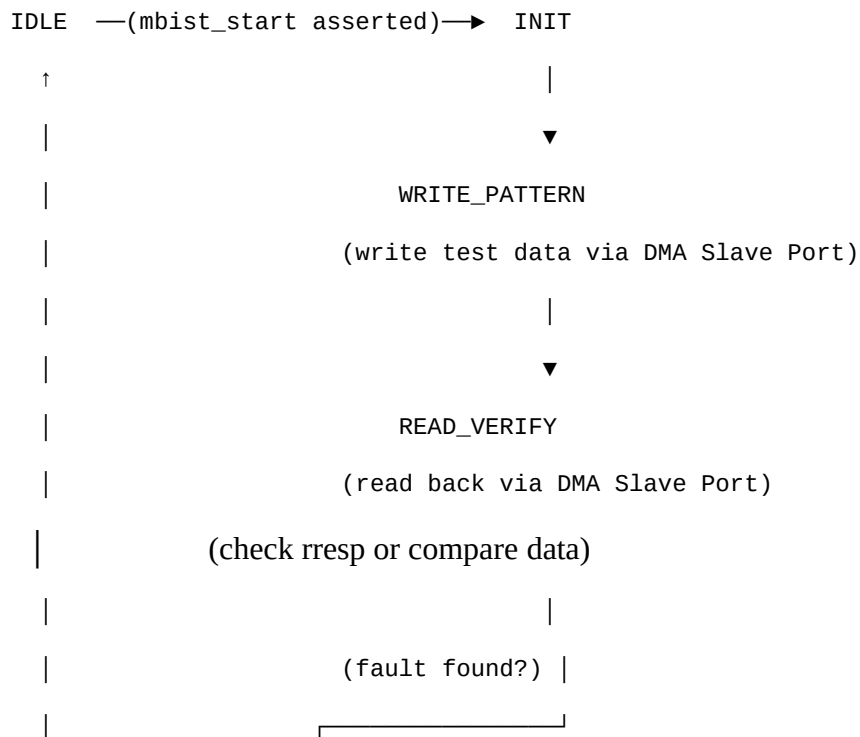
The Debug Module reset (`dbg_rst_l`) is a separate, independent active-low reset that resets the VeeR EL2 Debug Module and JTAG synchronisers. It may be tied to the SoC power-on reset to allow debugger interaction even while the core complex is in reset.

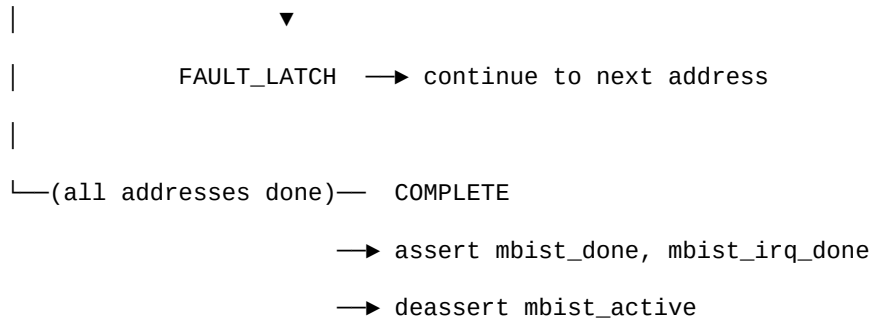
10.2.4 AXI4 Interconnect and Crossbar Design

The interconnect implements an AXI4 crossbar matrix supporting multiple masters (VeeR EL2 LSU, IFU, Debug, and MBIST Controller) and multiple targets (DCCM DMA port, system memories, and AXI4-Lite peripherals). Read and write address channels feature independent arbitration logic (round-robin with starvation prevention). An integrated protocol adapter bridges 64-bit AXI4 transactions to the 32-bit AXI4-Lite peripheral bus. Contention on the DMA Slave Port between the LSU and MBIST Controller is arbitrated according to the core's native priority rules (core accesses have priority, with periodic bubble allocation for DMA).

10.2.5 MBIST Controller State Machine

The MBIST Controller implements the following high-level state machine:





10.3 Firmware Design

Firmware is written in C targeting the RV32IMC ABI. The primary responsibilities of the firmware are:

- System initialisation (Section 8.2 sequence).
- Periodic MBIST result processing: read `mbist_pass_fail`, `mbist_fault_addr`, fault count; format and transmit via UART.
- Correctable-error ISR: read `mdccmect` via CSR instruction; write shadow to ECC Monitor registers.
- Watchdog pet: write `wdg_kick` at regular intervals within the main loop and within each ISR.
- GPIO update: set pass/fail LEDs based on `mbist_pass_fail` and `ecc_mon_status.mbist_correlated`.

Firmware accesses all peripheral registers through memory-mapped load/store instructions (standard C pointer dereference with volatile keyword). CSR accesses use inline assembly CSRR/CSRW instructions.

10.4 Tools and Environment

[To be completed with actual tool versions used in the project.]

Tool / Environment	Name	Version
RTL Simulation	ModelSim / Verilator / [tool name]	[version]
Synthesis	[Yosys / Synopsys DC / Cadence Genus]	[version]
RISC-V Toolchain	riscv32-unknown-elf-gcc (RV32IMC)	[version]
Assembler/Linker	GNU binutils for RISC-V	[version]
Version Control	Git	[version]
Documentation	Microsoft Word / Python-docx	[version]

Table 14b: Tools and Environment [Placeholder — complete with actual versions]

11. Verification and Test Plan

11.1 Verification Strategy

The verification strategy follows a hierarchical approach: each block is first verified independently with a dedicated unit testbench, then blocks are integrated progressively and tested at the subsystem and full-SoC levels. All test results described in this chapter are planned/expected; actual pass/fail results are to be recorded upon completion of simulation runs.

11.2 Testbench Architecture

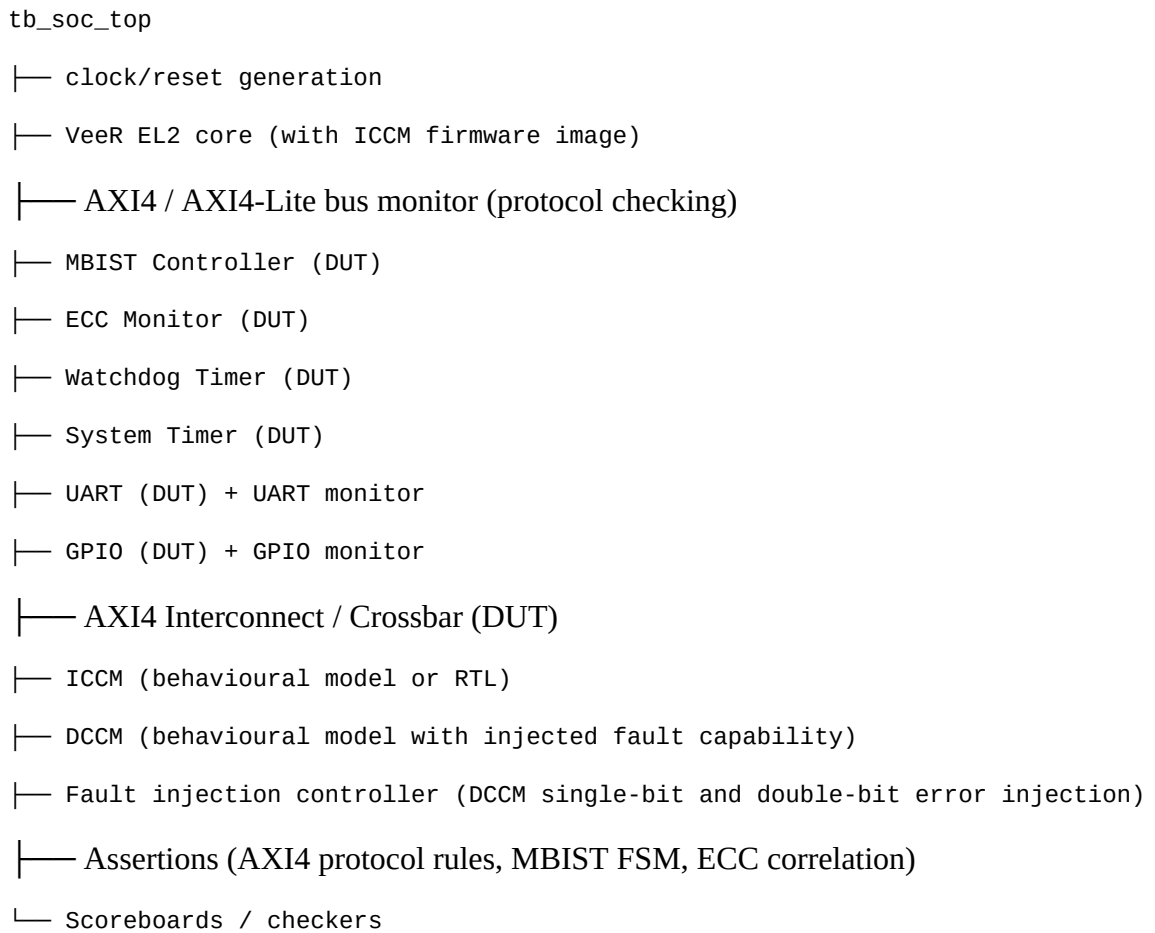


Figure 7 (planned): Testbench Architecture for Full-SoC Simulation

11.3 Unit Test Plan

The following unit testbenches are planned, one per major block:

Block	Test Coverage	Status
AXI4 Interconnect / Crossbar	Address decode correctness; valid/ready handshake compliance; unmapped address returns DECERR; arbitration	Planned

	between IFU/LSU/DMA; AXI4-Lite conversion	
MBIST Controller	Checkerboard and March C- AXI4 write/read burst/single sequences; double-bit error detection (inject rresp=SLVERR); address-decode fault detection; fault address and count registers; mbist_irq_done assertion	Planned
ECC Monitor	Shadow register write/read; mbist_active correlation; ecc_mon_suspect_addr population; ecc_mon_status flags; isolation (removal has no effect on MBIST)	Planned
Watchdog Timer	Countdown decrement; kick before timeout (no reset); no-kick timeout (wdg_timeout asserts); wdg_status latch across reset; wdg_irq_pretimeout timing	Planned
System Timer	Auto-reload; timer_tick pulse generation at programmed interval	Planned
UART	TX serialisation; RX deserialisation; baud rate configuration; framing error detection; busy/valid flags	Planned
GPIO	Direction register; output drive; input capture	Planned

Table 15: Unit Test Plan Summary

11.4 Integration Test Plan

Test Name	Description	Status
CPU-to-Peripheral Access	CPU reads/writes every mapped peripheral register via LSU AXI4 to AXI4-Lite bridge; response is OKAY; data is retained	Planned
Unmapped Access	Access to unmapped address completes with rresp/bresp=DECERR, rdata=0x00000000, no system crash or lockup	Planned
MBIST Full Scan — Pass	MBIST Controller runs March C- over full DCCM range with no injected faults; mbist_pass_fail=0; UART reports PASS	Planned
MBIST Double-Bit Fault	Inject double-bit stuck-at fault in DCCM; MBIST scan returns rresp=SLVERR; mbist_fault_addr matches injected address; mbist_fault_count=1	Planned
ECC Single-Bit Correlation	Inject single-bit stuck-at fault; trigger MBIST scan; verify mdccmect increments; verify ecc_mon_suspect_addr matches injected DCCM address range	Planned
Watchdog Recovery	Halt firmware (inhibit wdg_kick writes); verify wdg_timeout asserts; verify rst_l is re-asserted; verify wdg_status.timeout_occurred=1 after recovery	Planned
Timer-Triggered MBIST	Enable System Timer; verify timer_tick triggers mbist_start automatically at correct period; verify MBIST scan begins	Planned
Interrupt Routing	Each interrupt source (MBIST done, WDT pre-timeout, correctable-error) reaches the PIC and is handled by the correct ISR	Planned

Table 15b: Integration Test Plan

11.5 Expected Results

Note: No actual simulation results, waveforms, timing measurements, area figures, power figures, or coverage numbers are available at the time of writing. All results in this section are expected/planned outcomes based on the architectural specification. This section must be updated with actual results upon completion of verification.

11.5.1 MBIST Controller Expected Behaviour

- Checkerboard scan over 128 KB DCCM with no faults: using 64-bit AXI4 transfers, the scan covers 16,384 double-words, completing in approximately $2 \times 16,384 = 32,768$ DMA transactions (half the transaction count of a 32-bit bus); `mbist_pass_fail = 0`.
- March C- scan over 128 KB DCCM with no faults: using 64-bit AXI4 transfers, the scan completes in approximately $6 \times 16,384 = 98,304$ DMA transactions; `mbist_pass_fail = 0`.
- Double-bit fault injected at address A: `mbist_pass_fail = 1`; `mbist_fault_addr = A`; `mbist_fault_count  $\geq$  1`.
- Single-bit fault injected at address A: `mbist_pass_fail = 0` (MBIST cannot directly detect it); however, `mdccmect` increments during the scan and ECC Monitor sets `ecc_mon_suspect_addr` to a window containing A.

11.5.2 ECC Monitor Expected Behaviour

- With no injected faults and no MBIST scan: `ecc_mon_status = 0x00`; `ecc_mon_suspect_addr = 0x00000000`.
- With single-bit fault at address A and MBIST scan active over a window including A: `ecc_mon_status.correctable_seen = 1`; `ecc_mon_status.mbist_correlated = 1`; `ecc_mon_suspect_addr = MBIST window base address`.
- ECC Monitor removal: MBIST Controller and Watchdog Timer continue to function correctly.

11.5.3 Watchdog Timer Expected Behaviour

- Firmware pets the watchdog every N cycles ($N < \text{wdg_load}$): `wdg_timeout` remains de-asserted; system operates normally.
- Firmware fails to pet (simulated hang): `wdg_timeout` asserts after `wdg_load` cycles; `rst_l` is re-asserted; `wdg_status.timeout_occurred = 1` after reset.

11.6 Assertions

The following assertions are to be included in the testbench:

- AXI4: `AWVALID/WVALID/ARVALID` remain asserted until corresponding `READY` handshake is received.
- AXI4: `RRESP/BRESP = 2'b00` (OKAY) for normal access, `2'b10` (SLVERR) on uncorrectable ECC fault, `2'b11` (DECERR) on unmapped access.
- MBIST: `mbist_busy` is asserted throughout a scan and de-asserted on completion.
- MBIST: `mbist_active` tracks `mbist_busy` exactly.
- DCCM DMA write: writes are 32-bit or 64-bit aligned with valid byte strobes (`wstrb = 8'hFF` for 64-bit writes).
- ECC Monitor: `ecc_mon_suspect_addr` is only updated when `mbist_active = 1` at time of `shadow_count` increment.
- Watchdog: `wdg_timeout` asserts exactly `wdg_load` cycles after the last `wdg_kick` write.
- Interrupts: `mbist_irq_done` is asserted for exactly one cycle when `mbist_done` is set.

12. Limitations and Future Enhancements

12.1 Current Limitations

12.1.1 Single-Bit Fault Localisation

The ECC Monitor can identify the MBIST scan window (a range of DCCM addresses) that was active when a correctable-error interrupt fired, but it cannot identify the exact single DCCM word address of the fault within that window. The window granularity is determined by the `mbist_addr_start/end` configuration and the scan algorithm's progress at the time of the ECC event. Narrowing the window by running smaller MBIST scans improves localisation but increases scan time.

12.1.2 MBIST Operates on DCCM Only

The custom MBIST Controller targets DCCM via the DMA Slave Port. ICCM can also be accessed via the DMA Slave Port, but extending MBIST coverage to ICCM would require firmware to be stored elsewhere (external ROM or a shadow copy) during the ICCM scan, since the core cannot fetch instructions from ICCM while it is being overwritten by the MBIST write phase.

12.1.3 No Concurrent CPU and MBIST DCCM Access Ordering Guarantee

Per the VeeR EL2 PRM (Section 2.14.4), there are no ordering guarantees between core and DMA Slave Port accesses to the same DCCM address. Firmware must ensure that the DCCM region under test is not actively used by the CPU during an MBIST scan to prevent data corruption of live firmware variables.

12.1.4 MBIST Does Not Test During Live Firmware Execution

This design runs MBIST on a pre-configured DCCM address range. A full production-grade MBIST would typically require the core to be halted and the entire memory to be tested. Testing a live memory region requires partitioning the DCCM into a region in use by firmware and a region under test, and rotating through regions.

12.1.5 No Physical Synthesis Timing Closure

This project targets technology-independent synthesisable RTL. Physical timing closure, area optimisation, and power analysis at a specific technology node (e.g., 28 nm CMOS) are not performed. The 600 MHz VeeR EL2 target frequency is an ASIC target; FPGA or simulation runs at significantly lower frequencies.

12.1.6 ECC Monitor Relies on Firmware Interrupt Latency

The accuracy of the ECC correlation depends on firmware responding to the correctable-error local interrupt promptly and writing the `mdccmect` snapshot to the ECC Monitor before the

MBIST Controller advances to the next address window. If the interrupt latency is long relative to the MBIST scan speed, the suspect address window may be less precise.

12.2 Future Enhancements

12.2.1 ICCM MBIST Support

A future version could extend the MBIST Controller to test ICCM as well, with a dual-bank ICCM design that allows one bank to be tested while the other holds a minimal execution shadow for the core. This would enable full memory coverage without halting the system.

12.2.2 Improved Single-Bit Fault Localisation

An enhanced ECC Monitor could record the exact MBIST word address at the time of each correctable-error interrupt, rather than the entire scan window. This would require the MBIST Controller to expose a per-word address register (not just the window), and faster firmware interrupt service.

12.2.3 Adaptive MBIST Scheduling

Instead of fixed-period MBIST runs, an adaptive scheduler could increase scan frequency when the mdccmect count shows elevated error activity, enabling faster fault detection in ageing or degraded parts.

12.2.4 DCCM Scrubbing

For high-reliability applications, periodic DCCM scrubbing (reading and rewriting every word to correct and write back single-bit errors before they accumulate into double-bit faults) could be added as a firmware-driven background task scheduled by the System Timer.

12.2.5 Extension to Multi-Bank SRAMs

The architecture could be extended to SoCs with multiple independent SRAM banks by implementing one MBIST Controller per bank or a time-multiplexed MBIST Controller that addresses each bank in sequence.

12.2.6 Standardised MBIST Interface (IEEE 1500)

Integration with the IEEE 1500 Core Test Language (CTL) and a standardised MBIST wrapper would allow the custom MBIST Controller to be integrated into an ATPG / DFT flow for production test without modification.

13. Conclusion

This project has designed and specified a RISC-V based SoC for autonomous memory self-test and fault detection, using the VeeR EL2 core as the central processing element. The system addresses two closely related practical problems: the inability of conventional scan chains to test embedded memories in deployed systems, and the inability of SECDED ECC hardware alone to locate the specific DCCM cell address behind a developing hard fault.

The key architectural contribution is the combination of a custom MBIST Controller operating through the core's DMA Slave Port with a firmware-mediated ECC Monitor that correlates the core's CSR-resident correctable-error counter against the active MBIST scan window. This approach requires no modification to the VeeR EL2 core and no duplication of the core's native ECC hardware, while providing the suspect-address localisation capability that the core's own correctable-error counter alone cannot deliver.

A key architectural decision was the adoption of the high-performance AMBA AXI4 / AXI4-Lite protocol family throughout the SoC. Utilizing 64-bit AXI4 for memory and DMA transactions provides high test throughput and avoids AHB write turn-around bubbles, while implementing peripheral registers as native 32-bit AXI4-Lite slaves preserves low gate count and design modularity.

The Watchdog Timer provides a safety net for the entire self-test infrastructure: if the MBIST scan or any firmware component hangs, the system recovers automatically and records the event for post-mortem analysis—without requiring external intervention.

The design is specified at the RTL architecture level with complete signal lists, memory maps, register definitions, control and data path descriptions, and a detailed verification plan. Actual simulation results, timing numbers, and coverage data are to be completed during the implementation phase of the project.

In summary, the proposed SoC demonstrates that autonomous memory fault detection and ECC correlation can be achieved in a high-performance AXI4 architecture by leveraging the VeeR EL2 core's existing 64-bit DMA Slave Port and SECDED ECC hardware, combined with a minimal set of custom IPs and a disciplined firmware-mediated information relay.

References

- [1] CHIPS Alliance, "RISC-V VeeR EL2 Programmer's Reference Manual," Revision 1.4, December 22, 2022. Licensed under Apache-2.0. Available: <https://github.com/chipsalliance/Cores-VeeR-EL2>

- [2] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: User-Level ISA," Version 20190608-Base-Ratified, 2019. Available: <https://riscv.org/technical/specifications/>

- [3] RISC-V International, "The RISC-V Instruction Set Manual, Volume II: Privileged Architecture," Version 20190608-Priv-MSU-Ratified, 2019.

- [4] RISC-V International, "RISC-V External Debug Support," Version 0.13.2, 2019.

- [5] ARM Limited, "AMBA AXI and ACE Protocol Specification (AXI3, AXI4, and AXI4-Lite)," ARM IHI 0022E, 2013. Available: <https://developer.arm.com/documentation/ih0022/>

- [6] S. Hamdioui et al., "March Tests for All Static Simple RAM Faults: Theory and Practice," Proceedings of IOLTS, 2002.

- [7] A. J. van de Goor, "Testing Semiconductor Memories: Theory and Practice," John Wiley & Sons, 1991, ISBN 978-0-471-92986-0.

- [8] R. W. Hamming, "Error Detecting and Error Correcting Codes," Bell System Technical Journal, vol. 29, no. 2, pp. 147–160, April 1950.

- [9] CHIPS Alliance, "VeeR EL2 Core RTL Repository," GitHub, 2022. Available: <https://github.com/chipsalliance/Cores-VeeR-EL2>

- [10] IEEE Standard 1500-2005, "IEEE Standard Testability Method for Embedded Core-Based Integrated Circuits," IEEE, 2005.